

Extreme Programming (XP)

Extreme Programming (or XP) is a set of values, principles and practices for rapidly developing high-quality software that provides the highest value for the customer in the fastest way possible.

Characteristics

- It's not a methodology but a repeatable process for developing software
- Lightweight, low-ceremony, high-discipline methodology
- XP teams work in a series of fixed iteration cycles. Iterations typically last 1, 2 or 3 weeks.
- The system is always nearly defect-free.
- Up to a dozen programmers are recommended.
- Successful because it stresses customer satisfaction.
- At the end of the schedule, demonstrate the new software and see if the customer likes it.

Rules of Extreme Programming

Planning

- Each feature is written as a User Story, which includes a name and a description. "We just can't count on requirements being a constant. Instead, we will represent each feature the customer wants as a user story. Story cards are a low-cost mechanism for scoping out a project without spending our entire analysis budget up front." ¹
- Effort estimation for each user story and each iteration.
- Business then decides which stories to implement in what order.
- Small Releases: Release early and often.

Managing

- Dedicated open workspace for the team.
- Avoid working overtime.
- At the beginning of each iteration, the team gets together with the customer for a planning meeting.

- Determine the project's velocity (identify the capacity of the team to complete the tasks of the iteration, if tasks are not completed in time or finished early then adjust the assignments for next iteration).
- Rotate the roles, in that way everyone gets to know every part of the code and project.

Designing

- Keep it simple, do what's needed to meet today's requirements
- System Metaphor: naming conventions for projects, documents, classes, objects, database
- Spike Solutions: is something representing a risk for the project? Assign a pair of developers on the problem for a week or two.
- Keep design within the scope.
- Refactor whenever and wherever possible.

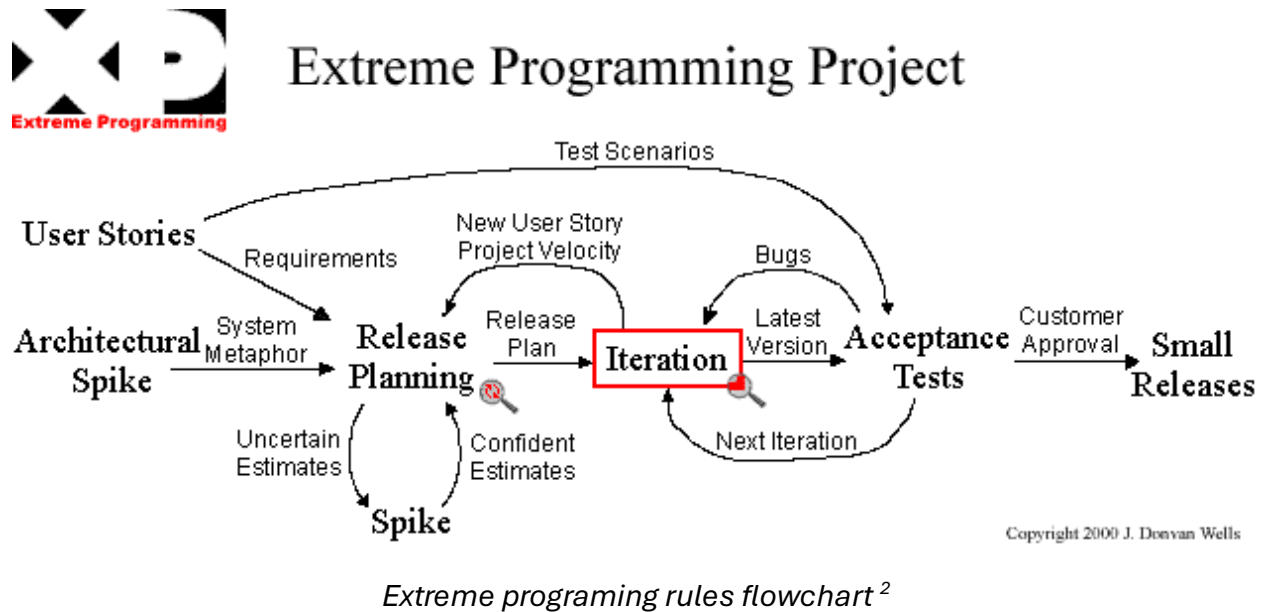
Coding

- User on the customer site who uses the system, always available.
- Everyone codes to the same standard.
- Write test suites first. Unit tests and Acceptance Tests.
- Code programming is performed by 2 people, pair programming.
- Only one person integrates code at a time and ensures all tests are successful
- Have a separate computer for integration which is always up to date and has no developments in progress
- Collective Code Ownership, anyone can work on any part of the code.

Testing

- All code must have tests and must pass before a release.
- When a bug is found, tests are created.
- Acceptance tests are executed often and the score published.

Flow Chart depicting the rules



XP Roles

Some roles can be combined in a single individual however XP proposes the following:

Tracker

- Checks progress once or twice a week
- Take actions like setting up a meeting with the customer or get help from someone else

Customer

- Write User Stories and specify Functional Tests (Acceptance tests)
- Job titles: Planner, Analyst, Project Lead, Product Lead, Project Manager or Designer
- Tells people what to do according to the plan

Programmer

- Estimate stories and identify tasks from stories
- Implement stories and junits

Coach

- Help with anything
- Support the team as mentor and methodologist.
- Communicate the vision of the team.

Tester

- Implement and run Functional Tests (Acceptance Tests)
- Make sure people know when a test fails after code integration

Doomsayer

- Red flag when something is not working out

Manager

- Schedule meetings
- Go to meetings with the customer and bring useful information
- Record meeting results (minutes)
- DOES NOT tell people what to do (Customer Role), when to be done or check progress (Tracker Role)

Citations

1 User stories quote. <http://www.agile-process.org/byfeature.html>

2 Diagram integrating XP rules. <http://www.extremeprogramming.org/map/project.html>